

Who This Book Is For

This book was written for practicing Python programmers who want to become proficient in Python 3. I tested the examples in Python 3.10—most of them also in Python 3.9 and 3.8. When an example requires Python 3.10, it should be clearly marked.

If you are not sure whether you know enough Python to follow along, review the topics of the official [Python tutorial](#). Topics covered in the tutorial will not be explained here, except for some features that are new.

Who This Book Is Not For

If you are just learning Python, this book is going to be hard to follow. Not only that, if you read it too early in your Python journey, it may give you the impression that every Python script should leverage special methods and metaprogramming tricks. Premature abstraction is as bad as premature optimization.

Five Books in One

I recommend that everyone read [Chapter 1](#), “[The Python Data Model](#)”. The core audience for this book should not have trouble jumping directly to any part in this book after [Chapter 1](#), but often I assume you’ve read preceding chapters in each specific part. Think of Parts [I](#) through [V](#) as books within the book.

I tried to emphasize using what is available before discussing how to build your own. For example, in [Part I](#), [Chapter 2](#) covers sequence types that are ready to use, including some that don’t get a lot of attention, like `collections.deque`. Building user-defined sequences is only addressed in [Part III](#), where we also see how to leverage the abstract base classes (ABCs) from `collections.abc`. Creating your own ABCs is discussed even later in [Part III](#), because I believe it’s important to be comfortable using an ABC before writing your own.

This approach has a few advantages. First, knowing what is ready to use can save you from reinventing the wheel. We use existing collection classes more often than we implement our own, and we can give more attention to the advanced usage of available tools by deferring the discussion on how to create new ones. We are also more likely to inherit from existing ABCs than to create a new ABC from scratch. And finally, I believe it is easier to understand the abstractions after you’ve seen them in action.

The downside of this strategy is the forward references scattered throughout the chapters. I hope these will be easier to tolerate now that you know why I chose this path.

Preface

Here’s the plan: when someone uses a feature you don’t understand, simply shoot them. This is easier than learning something new, and before too long the only living coders will be writing in an easily understood, tiny subset of Python 0.9.6 <wink>.¹

—Tim Peters, legendary core developer and author of *The Zen of Python*

“Python is an easy to learn, powerful programming language.” Those are the first words of the [official Python 3.10 tutorial](#). That is true, but there is a catch: because the language is easy to learn and put to use, many practicing Python programmers leverage only a fraction of its powerful features.

An experienced programmer may start writing useful Python code in a matter of hours. As the first productive hours become weeks and months, a lot of developers go on writing Python code with a very strong accent carried from languages learned before. Even if Python is your first language, often in academia and in introductory books it is presented while carefully avoiding language-specific features.

As a teacher introducing Python to programmers experienced in other languages, I see another problem that this book tries to address: we only miss stuff we know about. Coming from another language, anyone may guess that Python supports regular expressions, and look that up in the docs. But if you’ve never seen tuple unpacking or descriptors before, you will probably not search for them, and you may end up not using those features just because they are specific to Python.

This book is not an A-to-Z exhaustive reference of Python. Its emphasis is on the language features that are either unique to Python or not found in many other popular languages. This is also mostly a book about the core language and some of its libraries. I will rarely talk about packages that are not in the standard library, even though the Python package index now lists more than 60,000 libraries, and many of them are incredibly useful.

¹ Message to the comp.lang.python Usenet group, Dec. 23, 2002: “[Acrimony in c.l.p](#)”.